

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)*

Activity Tool : Code Challenge (High)

Assessment Tool Weightage: 35%

Faculty Name: Dr.G.Ayyappan

Student Reg. No: 192424061

Name: K.Sandhya

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Write a Python/C program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.	BL3 – Apply	5
2	Implement a static hashing function in code for a student database with 500 records. Handle collisions using chaining.	BL3 – Apply	5
3	Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.	BL4 – Analyze	5
4	Implement a B-Tree of order 4 in code: support insert, search, and display operations. Insert keys: 10, 20, 5, 6, 12, 30, 7, 17.	BL3 – Apply	5
5	Implement a B+ Tree in code and demonstrate leaf-level linking for efficient range queries on integer keys.	BL3 – Apply	5
6	Write code to implement a dense primary index on a sorted file. Support search by index key and display the index table.	BL4 – Analyze	5
7	Implement extendible hashing in code. Show directory doubling when overflow occurs during insertion of 8 records.	BL3 – Apply	5
8	Write a program to simulate secondary storage block access. Calculate the number of I/O operations for sequential vs. indexed retrieval.	BL4 – Analyze	5

9	Implement a sparse index on a sorted file in code and compare its search performance with a dense index using timing analysis.	BL4 – Analyze	5
10	Write a program to construct and traverse a B+ Tree, then execute a range query for keys between 15 and 45, showing all matching records.	BL4 – Analyze	5

Code of Conduct:

*I _K.Sandhya 192424061_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Question 1 : Write a Python/C program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.

THEORY

A Heap File stores records in no particular order. New records are usually inserted at the end or in any available space. It provides simple and fast insertion, but searching for a particular record may require scanning the entire file.

AIM

To simulate a Heap File organization and perform insert, delete, and sequential scan operations.

ALGORITHM

1. Create an empty heap file.
2. Insert records into the file.
3. To delete, search for the required record and remove it.
4. Perform sequential scanning from the first record to the last.
5. Display all available records.

ADVANTAGES

- Simple implementation.
- Fast insertion.
- Suitable when records are frequently inserted.

DISADVANTAGE

- Searching is slow because records are unordered.

CODE :

```
# Heap File Organization
```

```
heap = []

def insert(record):

    heap.append(record)

    print("Record inserted:", record)

def delete(record_id):

    for record in heap:

        if record[0] == record_id:

            heap.remove(record)

            print("Record deleted:", record)

            return

    print("Record not found")

def sequential_scan():

    print("\nHeap File Records:")

    if not heap:

        print("No records found")

    else:

        for record in heap:

            print(record)

while True:

    print("\n--- Heap File Organization ---")
```

```
print("1. Insert")

print("2. Delete")

print("3. Sequential Scan")

print("4. Exit")

choice = int(input("Enter your choice: "))

if choice == 1:

    record_id = int(input("Enter Record ID: "))

    name = input("Enter Name: ")

    record = (record_id, name)

    insert(record)

elif choice == 2:

    record_id = int(input("Enter Record ID to delete: "))

    delete(record_id)

elif choice == 3:

    sequential_scan()

elif choice == 4:

    print("Program terminated.")

    break

else:

    print("Invalid choice")
```

OUTPUT:

```
--- Heap File Organization ---
1. Insert
2. Delete
3. Sequential Scan
4. Exit
Enter your choice: 1
Enter Record ID: 101
Enter Name: arun
Record inserted: (101, 'arun')

--- Heap File Organization ---
1. Insert
2. Delete
3. Sequential Scan
4. Exit
Enter your choice: 1
Enter Record ID: 102
Enter Name: priya
Record inserted: (102, 'priya')

--- Heap File Organization ---
1. Insert
2. Delete
3. Sequential Scan
4. Exit
Enter your choice: 3

Heap File Records:
(101, 'arun')
(102, 'priya')
```

Question 2 : Implement a static hashing function in code for a student database with 500 records. Handle collisions using chaining.

Aim

To implement a static hashing function for a student database containing 500 records, using chaining to handle collisions.

Algorithm

1. Create a hash table with a fixed number of buckets.
2. Use the hash function:
$$\text{index} = \text{Student_ID} \% \text{table_size}$$
3. Insert each student record into the calculated bucket.
4. If multiple records have the same hash index, store them in the same bucket using a list (chaining).
5. Search for a student using the same hash function.
6. Display the hash table to show the chained records.

CODE:

Static Hashing with Chaining

Student Database - 500 Records

```
TABLE_SIZE = 50
```

```
# Create hash table
```

```
hash_table = [[] for _ in range(TABLE_SIZE)]
```

```
# Hash function
```

```
def hash_function(student_id):
```

```
    return student_id % TABLE_SIZE
```

```
# Insert student record
```

```
def insert(student_id, name, department):
```

```
    index = hash_function(student_id)
```

```
    # Add record to the bucket
```

```
    hash_table[index].append(
```

```
        (student_id, name, department)
```

```
    )
```

```
    print("Student inserted successfully.")
```

```
# Search student record
```

```
def search(student_id):
```

```
    index = hash_function(student_id)
```

```
    for record in hash_table[index]:
```

```
        if record[0] == student_id:
```



```

        print("Student found:")

        print("ID:", record[0])

        print("Name:", record[1])

        print("Department:", record[2])

        return

    print("Student not found.")

# Delete student record

def delete(student_id):

    index = hash_function(student_id)

    for record in hash_table[index]:

        if record[0] == student_id:

            hash_table[index].remove(record)

            print("Student deleted successfully.")

            return

    print("Student not found.")

# Display hash table

def display():

    print("\n--- Hash Table ---")

    for i in range(TABLE_SIZE):

        print("Bucket", i, ":", hash_table[i])

```

```
# Menu
```

```
while True:
```

```
    print("\n--- Static Hashing ---")
```

```
    print("1. Insert Student")
```

```
    print("2. Search Student")
```

```
    print("3. Delete Student")
```

```
    print("4. Display Hash Table")
```

```
    print("5. Exit")
```

```
    choice = int(input("Enter your choice: "))
```

```
    if choice == 1:
```

```
        student_id = int(input("Enter Student ID: "))
```

```
        name = input("Enter Student Name: ")
```

```
        department = input("Enter Department: ")
```

```
        insert(student_id, name, department)
```

```
    elif choice == 2:
```

```
        student_id = int(input("Enter Student ID to search: "))
```

```
        search(student_id)
```

```
    elif choice == 3:
```

```
        student_id = int(input("Enter Student ID to delete: "))
```

```
        delete(student_id)
```

```

elif choice == 4:

    display()

elif choice == 5:

    print("Program terminated.")

    break

else:

    print("Invalid choice.")

```

OUTPUT:

```

--- Static Hashing ---
1. Insert Student
2. Search Student
3. Delete Student
4. Display Hash Table
5. Exit
Enter your choice: 1
Enter Student ID: 101
Enter Student Name: ABON
Enter Department: CSE
Student inserted successfully.

--- Static Hashing ---
1. Insert Student
2. Search Student
3. Delete Student
4. Display Hash Table
5. Exit
Enter your choice: 1
Enter Student ID: 151
Enter Student Name: PRIYA
Enter Department: IT
Student inserted successfully.

--- Static Hashing ---
1. Insert Student
2. Search Student
3. Delete Student
4. Display Hash Table
5. Exit
Enter your choice: 4

--- Hash Table ---
Bucket 0 : []
Bucket 1 : [(101, 'ABON', 'CSE')]
Bucket 2 : []
Bucket 3 : []
Bucket 4 : []
Bucket 5 : []
Bucket 6 : []
Bucket 7 : []
Bucket 8 : []
Bucket 9 : []
Bucket 10 : []
Bucket 11 : []
Bucket 12 : []
Bucket 13 : []
Bucket 14 : []
Bucket 15 : [(151, 'PRIYA', 'IT')]
Bucket 16 : []
Bucket 17 : []

```

Question 3 : Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.

Aim

To simulate a Sorted File Organization in Python and implement Binary Search for retrieving records using the primary key.

Algorithm

1. Create a list of student records sorted by Student_ID.
2. Insert records while maintaining sorted order.

3. Use binary search to find the required Student_ID.
4. Compare the search key with the middle record.
5. If equal, display the record.
6. If the key is smaller, search the left half.
7. If the key is larger, search the right half.
8. Repeat until the record is found or the search range becomes empty.

CODE :

INPUT:

Sorted File Organization with Binary Search

records = [

(101, "Arun", "CSE"),

(105, "Priya", "IT"),

(110, "Rahul", "ECE"),

(115, "Anu", "CSE"),

(120, "Kiran", "MECH")

]

Display sorted records

def display_records():

print("\n--- Sorted File Records ---")

for record in records:

print("ID:", record[0],

"Name:", record[1],

```
"Department:", record[2])
```

```
# Binary Search
```

```
def binary_search(key):
```

```
    low = 0
```

```
    high = len(records) - 1
```

```
    while low <= high:
```

```
        mid = (low + high) // 2
```

```
        if records[mid][0] == key:
```

```
            return records[mid]
```

```
        elif key < records[mid][0]:
```

```
            high = mid - 1
```

```
        else:
```

```
            low = mid + 1
```

```
    return None
```

```
# Insert record in sorted order
```

```
def insert_record(record):
```

```
    records.append(record)
```

```
    records.sort(key=lambda x: x[0])
```

```
    print("Record inserted successfully.")
```

```
# Menu
```

```
while True:

    print("\n--- Sorted File Organization ---")

    print("1. Display Records")

    print("2. Binary Search")

    print("3. Insert Record")

    print("4. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:

        display_records()

    elif choice == 2:

        key = int(input("Enter Student ID to search: "))

        result = binary_search(key)

        if result:

            print("\nRecord Found")

            print("Student ID:", result[0])

            print("Name:", result[1])

            print("Department:", result[2])

        else:

            print("Record not found.")
```

```

elif choice == 3:

    student_id = int(input("Enter Student ID: "))

    name = input("Enter Name: ")

    department = input("Enter Department: ")

    insert_record((student_id, name, department))

elif choice == 4:

    print("Program terminated.")

    break

else:

    print("Invalid choice.")

```

OUTPUT :

```

--- Sorted File Organization ---
1. Display Records
2. Binary Search
3. Insert Record
4. Exit
Enter your choice: 1

--- Sorted File Records ---
ID: 101 Name: Arun Department: CSE
ID: 105 Name: Priya Department: IT
ID: 110 Name: Rahul Department: ECE
ID: 115 Name: Anu Department: CSE
ID: 120 Name: Kiran Department: MECH

--- Sorted File Organization ---
1. Display Records
2. Binary Search
3. Insert Record
4. Exit
Enter your choice: 2
Enter Student ID to search: 115

Record Found
Student ID: 115
Name: Anu
Department: CSE

```

Question 4 : Implement a B-Tree of order 4 in code: support insert, search, and display operations. Insert keys: 10, 20, 5, 6, 12, 30, 7, 17.

Aim

To implement a B-Tree of order 4 in Python with insert, search, and display operations using the keys 10, 20, 5, 6, 12, 30, 7, 17.

Algorithm

1. Create a B-Tree of order 4.
2. Each node can contain a maximum of 3 keys and 4 children.
3. Insert each key into the appropriate position.
4. If a node becomes full, split it and move the middle key to the parent.
5. For searching, compare the key with keys in the current node and move to the appropriate child.
6. Display the tree level by level.

Code :

Input:

```
# B-Tree of Order 4
```

```
# Maximum 3 keys per node
```

```
class BTreeNode:
```

```
    def __init__(self, leaf=True):
```

```
        self.keys = []
```

```
        self.children = []
```

```
        self.leaf = leaf
```

```
class BTree:
```

```
    def __init__(self):
```

```
        self.root = BTreeNode(True)
```


Search operation

```
def search(self, node, key):
```

```
    i = 0
```

```
    while i < len(node.keys) and key > node.keys[i]:
```

```
        i += 1
```

```
    if i < len(node.keys) and key == node.keys[i]:
```

```
        return True
```

```
    if node.leaf:
```

```
        return False
```

```
    return self.search(node.children[i], key)
```

Split a full child

```
def split_child(self, parent, index):
```

```
    full_child = parent.children[index]
```

```
    new_child = BTreeNode(full_child.leaf)
```

```
    # Middle key
```

```
    middle_key = full_child.keys[1]
```

```
# Move right key to new child
```

```
new_child.keys = full_child.keys[2:]
```

```
# Keep left key in old child
```

```
full_child.keys = full_child.keys[:1]
```

```
# Move children if not leaf
```

```
if not full_child.leaf:
```

```
    new_child.children = full_child.children[2:]
```

```
    full_child.children = full_child.children[:2]
```

```
# Add new child and middle key to parent
```

```
parent.children.insert(index + 1, new_child)
```

```
parent.keys.insert(index, middle_key)
```

```
# Insert into a non-full node
```

```
def insert_non_full(self, node, key):
```

```
    i = len(node.keys) - 1
```

```
    if node.leaf:
```

```
        node.keys.append(0)
```

```
        while i >= 0 and key < node.keys[i]:
```

```
            node.keys[i + 1] = node.keys[i]
```

```

        i -= 1

        node.keys[i + 1] = key

    else:

        while i >= 0 and key < node.keys[i]:

            i -= 1

        i += 1

        # Split child if full

        if len(node.children[i].keys) == 3:

            self.split_child(node, i)

            if key > node.keys[i]:

                i += 1

            self.insert_non_full(node.children[i], key)

# Insert operation

def insert(self, key):

    root = self.root

    # If root is full, create a new root

    if len(root.keys) == 3:

        new_root = BTreeNode(False)

```

```

        new_root.children.append(root)

        self.root = new_root

        self.split_child(new_root, 0)

        self.insert_non_full(new_root, key)

    else:

        self.insert_non_full(root, key)

# Display operation

def display(self, node=None, level=0):

    if node is None:

        node = self.root

    print("Level", level, ":", node.keys)

    if not node.leaf:

        for child in node.children:

            self.display(child, level + 1)

# Create B-Tree

btree = BTree()

# Insert given keys

keys = [10, 20, 5, 6, 12, 30, 7, 17]

```

```
for key in keys:

    btree.insert(key)

# Display B-Tree

print("B-Tree of Order 4:")

btree.display()

# Search operation

key = int(input("\nEnter key to search: "))

if btree.search(btree.root, key):

    print("Key", key, "found in B-Tree.")

else:

    print("Key", key, "not found in B-Tree.")
```

Output:

```
B-Tree of Order 4:
Level 0 : [10, 20]
Level 1 : [5, 6, 7]
Level 1 : [12, 17]
Level 1 : [30]

Enter key to search: 17
Key 17 found in B-Tree.
```

Question 5 : Implement a B+ Tree in code and demonstrate leaf-level linking for efficient range queries on integer keys

Aim

To implement a B+ Tree in Python and demonstrate leaf-level linking for efficient range queries on integer keys.

Algorithm

1. Create B+ Tree nodes with keys and child pointers.
2. Insert integer keys into the appropriate leaf node.
3. If a leaf becomes full, split it into two leaves.
4. Link the new leaf with the previous leaf using a next pointer.
5. Promote the separator key to the parent node.
6. For searching, traverse from the root to the required leaf.
7. For a range query, find the starting leaf and follow the linked leaf nodes until the upper limit is reached

CODE :

INPUT:

B+ Tree Implementation

Integer keys with leaf-level linking

ORDER = 4

class Node:

def __init__(self, leaf=False):

self.keys = []

```
self.children = []
```

```
self.leaf = leaf
```

```
self.next = None
```

```
class BPlusTree:
```

```
def __init__(self):
```

```
    self.root = Node(leaf=True)
```

```
# Find the leaf node for a key
```

```
def find_leaf(self, key):
```

```
    current = self.root
```

```
    while not current.leaf:
```

```
        i = 0
```

```
        while i < len(current.keys) and key >= current.keys[i]:
```

```
            i += 1
```

```
        current = current.children[i]
```

```
    return current
```

```
# Insert a key
```

```
def insert(self, key):
```

```
    root = self.root
```

```

# Root is a leaf

if root.leaf:

    if key not in root.keys:

        root.keys.append(key)

        root.keys.sort()

    if len(root.keys) >= ORDER:

        self.split_leaf(root)

    return

leaf = self.find_leaf(key)

if key in leaf.keys:

    return

leaf.keys.append(key)

leaf.keys.sort()

if len(leaf.keys) >= ORDER:

    self.split_leaf(leaf)

# Split leaf node

def split_leaf(self, leaf):

    new_leaf = Node(leaf=True)

```



```

mid = len(leaf.keys) // 2

new_leaf.keys = leaf.keys[mid:]

leaf.keys = leaf.keys[:mid]

# Link leaves

new_leaf.next = leaf.next

leaf.next = new_leaf

# If leaf is root

if leaf == self.root:

    new_root = Node()

    new_root.keys = [new_leaf.keys[0]]

    new_root.children = [leaf, new_leaf]

    self.root = new_root

else:

    self.insert_into_parent(leaf, new_leaf.keys[0], new_leaf)

# Insert separator key into parent

def insert_into_parent(self, old_node, key, new_node):

    parent = self.find_parent(self.root, old_node)

```

```

index = parent.children.index(old_node)

parent.keys.insert(index, key)

parent.children.insert(index + 1, new_node

# Split parent if necessary

if len(parent.keys) >= ORDER:

    self.split_internal(parent)

# Find parent of a node

def find_parent(self, current, target):

    if current.leaf:

        return None

    for child in current.children:

        if child == target:

            return current

        if not child.leaf:

            result = self.find_parent(child, target)

            if result:

                return result

```

```
return None
```

```
# Split internal node
```

```
def split_internal(self, node):
```

```
    new_node = Node()
```

```
    mid = len(node.keys) // 2
```

```
    promoted_key = node.keys[mid]
```

```
    new_node.keys = node.keys[mid + 1:]
```

```
    new_node.children = node.children[mid + 1:]
```

```
    node.keys = node.keys[:mid]
```

```
    node.children = node.children[:mid + 1]
```

```
# If node is root
```

```
if node == self.root:
```

```
    new_root = Node()
```

```
    new_root.keys = [promoted_key]
```

```
    new_root.children = [node, new_node]
```

```
    self.root = new_root
```

```
else:
```

```
    self.insert_into_parent(node, promoted_key, new_node)
```

```
# Search operation
```

```
def search(self, key):
```

```
    leaf = self.find_leaf(key)
```

```
    if key in leaf.keys:
```

```
        return True
```

```
    return False
```

```
# Range query using linked leaves
```

```
def range_query(self, start, end):
```

```
    leaf = self.find_leaf(start)
```

```
    result = []
```

```
    while leaf:
```

```
        for key in leaf.keys:
```

```
            if key > end:
```

```
                return result
```

```
            if key >= start:
```

```
                result.append(key)
```

```

        leaf = leaf.next

    return result

# Display tree

def display(self, node=None, level=0):

    if node is None:

        node = self.root

    print("Level", level, ":", node.keys)

    if not node.leaf:

        for child in node.children:

            self.display(child, level + 1)

# Display linked leaves

def display_leaves(self):

    leaf = self.root

    while not leaf.leaf:

        leaf = leaf.children[0]

    print("\nLeaf Level Linking:")

    while leaf:

        print(leaf.keys, end=" -> ")

```

```
leaf = leaf.next

    print("None")

# -----

# Main Program

# -----

tree = BPlusTree()

keys = [10, 20, 5, 6, 12, 30, 7, 17, 25, 40, 15]

print("Inserting keys:")

for key in keys:

    print(key, end=" ")

    tree.insert(key)

print("\n\nB+ Tree:")

tree.display()

tree.display_leaves()

# Search

key = int(input("\nEnter key to search: "))

if tree.search(key):

    print("Key", key, "found.")
```

else:

```
print("Key", key, "not found.")
```

Range Query

```
start = int(input("\nEnter range start: "))
```

```
end = int(input("Enter range end: "))
```

```
result = tree.range_query(start, end)
```

```
print("Keys in range:", result)
```

OUTPUT :

```
Inserting keys:
10 20 5 6 12 30 7 17 25 40 15

B+ Tree:
Level 0 : [20]
Level 1 : [10, 15]
Level 2 : [5, 6, 7]
Level 2 : [10, 12]
Level 2 : [15, 17]
Level 1 : [30]
Level 2 : [20, 25]
Level 2 : [30, 40]

Leaf Level Linking:
[5, 6, 7] -> [10, 12] -> [15, 17] -> [20, 25] -> [30, 40] -> None

Enter key to search: 17
Key 17 found.

Enter range start: 10
Enter range end: 30
Keys in range: [10, 12, 15, 17, 20, 25, 30]
```

Question 6 : Write code to implement a dense primary index on a sorted file. Support search by index key and display the index table.

Aim

To implement a Dense Primary Index on a sorted file and support search by index key and display of the index table.

Algorithm

1. Create a sorted file containing student records.
2. Create an index entry for every record.
3. Store the primary key and its corresponding record position in the index.
4. For searching, find the key in the index.
5. Use the stored position to retrieve the corresponding record.
6. Display the complete index table

Code :

Input:

Dense Primary Index on a Sorted File

Sorted file records

records = [

(101, "Arun", "CSE"),

(105, "Priya", "IT"),

(110, "Rahul", "ECE"),

(115, "Anu", "CSE"),

(120, "Kiran", "MECH")

]

Create dense primary index

Every record has an index entry

index_table = []


```
for position, record in enumerate(records):

    index_table.append((record[0], position))

# Display index table

def display_index():

    print("\n--- Dense Primary Index ---")

    print("Index Key\tRecord Position")

    print("-----")

    for key, position in index_table:

        print(key, "\t\t", position)

# Search using index

def search(key):

    for index_key, position in index_table:

        if index_key == key:

            print("\nRecord Found")

            print("Index Key:", index_key)

            print("Record Position:", position)

            record = records[position]

            print("Student ID:", record[0])
```

```
        print("Name:", record[1])

        print("Department:", record[2])

        return

    print("\nRecord not found.")

# Main program

while True:

    print("\n--- Dense Primary Index ---")

    print("1. Display Index")

    print("2. Search Record")

    print("3. Display Sorted File")

    print("4. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:

        display_index()

    elif choice == 2:

        key = int(input("Enter Student ID to search: "))

        search(key)

    elif choice == 3:
```

```

print("\n--- Sorted File ---")

for position, record in enumerate(records):

    print(position, ":", record)

elif choice == 4:

    print("Program terminated.")

    break

else:

    print("Invalid choice.")

```

OUTPUT :

```

--- Dense Primary Index ---
1. Display Index
2. Search Record
3. Display Sorted File
4. Exit
Enter your choice: 2
Enter Student ID to search: 115

Record Found
Index Key: 115
Record Position: 3
Student ID: 115
Name: Anu
Department: CSE

```

Question 7 : Implement extendible hashing in code. Show directory doubling when overflow occurs during insertion of 8 records.

Aim

To implement Extendible Hashing in Python and demonstrate directory doubling when overflow occurs during the insertion of 8 records.

Algorithm

1. Create a directory with an initial global depth of 1.
2. Each bucket has a fixed capacity.
3. Use the binary representation of the key for hashing.
4. Insert records into the bucket determined by the directory bits.
5. When a bucket overflows:
 - If local depth is less than global depth, split the bucket.
 - If local depth equals global depth, double the directory and then split the bucket.
6. Update the directory pointers after splitting.
7. Continue insertion until all 8 records are inserted.
8. Display the directory after each insertion.

Code :

Input:

Extendible Hashing

Demonstrates directory doubling during insertion

BUCKET_SIZE = 2

class Bucket:

def __init__(self, depth):

self.local_depth = depth

self.records = []

class ExtendibleHashing:

def __init__(self):

self.global_depth = 1

```

# Initial two buckets

self.directory = [

    Bucket(1),

    Bucket(1)

]

# Hash function

def hash_function(self, key):

    return key

# Get directory index

def get_index(self, key):

    hash_value = self.hash_function(key)

    mask = (1 << self.global_depth) - 1

    return hash_value & mask

# Double directory

def double_directory(self):

    old_size = len(self.directory)

    # Duplicate directory pointers

    self.directory = self.directory + self.directory.copy()

    self.global_depth += 1

    print("\n*** DIRECTORY DOUBLED ***")

```

```

    print("New Global Depth:", self.global_depth)

# Split bucket

def split_bucket(self, index):

    old_bucket = self.directory[index]

    old_depth = old_bucket.local_depth

    # Increase local depth

    new_depth = old_depth + 1

    old_bucket.local_depth = new_depth

    new_bucket = Bucket(new_depth)

    # Find pattern for new bucket

    split_bit = 1 << (new_depth - 1)

    # Update directory pointers

    for i in range(len(self.directory)):

        if self.directory[i] is old_bucket:

            if i & split_bit:

                self.directory[i] = new_bucket

    # Reinsert old records

    old_records = old_bucket.records

    old_bucket.records = []

    for record in old_records:

```

```
new_index = self.get_index(record)

self.directory[new_index].records.append(record)

# Insert record

def insert(self, key):

    while True:

        index = self.get_index(key)

        bucket = self.directory[index]

        # No overflow

        if len(bucket.records) < BUCKET_SIZE:

            bucket.records.append(key)

            print("Inserted:", key)

            return

        # Bucket overflow

        print("\nOverflow occurred for key:", key)

        # Directory needs doubling

        if bucket.local_depth == self.global_depth:

            self.double_directory()

        # Split bucket

        self.split_bucket(index)

# Display directory
```

```

def display(self):

    print("\n--- Extendible Hash Directory ---")

    print("Global Depth:", self.global_depth)

    for i, bucket in enumerate(self.directory):

        binary_index = format(i, f"0{self.global_depth}b")

        print(

            binary_index,

            "->",

            bucket.records,

            "(Local Depth:",

            bucket.local_depth,

            ")"

# -----

# Main Program

# -----

eh = ExtendibleHashing()

# Insert 8 records

records = [1, 3, 5, 7, 9, 11, 13, 15]

print("Inserting 8 records...\n")

for record in records:

```



```
eh.insert(record)
```

```
eh.display()
```

```
print("\nFinal Extendible Hashing Structure:")
```

```
eh.display()
```

Output:

```
Inserting 8 records...

Inserted: 1

--- Extendible Hash Directory ---
Global Depth: 1
0 -> [] (Local Depth: 1 )
1 -> [1] (Local Depth: 1 )
Inserted: 3

--- Extendible Hash Directory ---
Global Depth: 1
0 -> [] (Local Depth: 1 )
1 -> [1, 3] (Local Depth: 1 )

Overflow occurred for key: 5

*** DIRECTORY DOUBLED ***
New Global Depth: 2
Inserted: 5

--- Extendible Hash Directory ---
Global Depth: 2
00 -> [] (Local Depth: 1 )
01 -> [1, 5] (Local Depth: 2 )
10 -> [] (Local Depth: 1 )
11 -> [3] (Local Depth: 2 )
Inserted: 7

--- Extendible Hash Directory ---
Global Depth: 2
00 -> [] (Local Depth: 1 )
01 -> [1, 5] (Local Depth: 2 )
10 -> [] (Local Depth: 1 )
11 -> [3, 7] (Local Depth: 2 )

Overflow occurred for key: 9

--- Extendible Hash Directory ---
Global Depth: 3
000 -> [] (Local Depth: 1 )
001 -> [1, 9] (Local Depth: 3 )
010 -> [] (Local Depth: 1 )
011 -> [3, 7] (Local Depth: 2 )
100 -> [] (Local Depth: 1 )
101 -> [5] (Local Depth: 3 )
110 -> [] (Local Depth: 1 )
111 -> [3, 7] (Local Depth: 2 )

Overflow occurred for key: 11
Inserted: 11

--- Extendible Hash Directory ---
Global Depth: 3
000 -> [] (Local Depth: 1 )
001 -> [1, 9] (Local Depth: 3 )
010 -> [] (Local Depth: 1 )
011 -> [3, 11] (Local Depth: 3 )
100 -> [] (Local Depth: 1 )
101 -> [5] (Local Depth: 3 )
110 -> [] (Local Depth: 1 )
111 -> [7] (Local Depth: 3 )
Inserted: 13

--- Extendible Hash Directory ---
Global Depth: 3
000 -> [] (Local Depth: 1 )
001 -> [1, 9] (Local Depth: 3 )
010 -> [] (Local Depth: 1 )
011 -> [3, 11] (Local Depth: 3 )
100 -> [] (Local Depth: 1 )
101 -> [5, 13] (Local Depth: 3 )
110 -> [] (Local Depth: 1 )
111 -> [7] (Local Depth: 3 )
Inserted: 15
```

```
--- Extendible Hash Directory ---
Global Depth: 3
000 -> [] (Local Depth: 1 )
001 -> [1, 9] (Local Depth: 3 )
010 -> [] (Local Depth: 1 )
011 -> [3, 11] (Local Depth: 3 )
100 -> [] (Local Depth: 1 )
101 -> [5, 13] (Local Depth: 3 )
110 -> [] (Local Depth: 1 )
111 -> [7, 15] (Local Depth: 3 )

Final Extendible Hashing Structure:

--- Extendible Hash Directory ---
Global Depth: 3
000 -> [] (Local Depth: 1 )
001 -> [1, 9] (Local Depth: 3 )
010 -> [] (Local Depth: 1 )
011 -> [3, 11] (Local Depth: 3 )
100 -> [] (Local Depth: 1 )
101 -> [5, 13] (Local Depth: 3 )
110 -> [] (Local Depth: 1 )
111 -> [7, 15] (Local Depth: 3 )
>>>|
```

Question 8 : Write a program to simulate secondary storage block access. Calculate the number of I/O operations for sequential vs. indexed retrieval.

Aim

To simulate secondary storage block access and compare the number of I/O operations required for sequential retrieval and indexed retrieval.

Algorithm

1. Create a list of records stored in fixed-size blocks.
2. Simulate sequential retrieval by reading blocks one by one until the required record is found.
3. Create an index containing the key and corresponding block number.
4. Simulate indexed retrieval by first accessing the index and then directly accessing the required data block.
5. Count and display the I/O operations for both methods.
6. Compare the results.

Code :

Input:

Secondary Storage Block Access Simulation

Records stored in secondary storage

records = [

(101, "Alice"),

(102, "Bob"),

(103, "Charlie")

```
(104, "David"),
```

```
(105, "Eva"),
```

```
(106, "Frank"),
```

```
(107, "George"),
```

```
(108, "Helen"
```

```
# Number of records in each block
```

```
block_size = 2
```

```
# Create blocks
```

```
blocks = []
```

```
for i in range(0, len(records), block_size):
```

```
    blocks.append(records[i:i + block_size])
```

```
print("Secondary Storage Blocks:")
```

```
for i, block in enumerate(blocks, start=1):
```

```
    print("Block", i, ":", block)
```

```
# Sequential Retrieval
```

```
def sequential_retrieval(key):
```

```
    io_operations = 0
```

```
for block in blocks:
```

```
    io_operations += 1    # Read one block
```

```
    for record in block:
```

```
        if record[0] == key:
```

```
            print("\nSequential Retrieval")
```

```
            print("Record found:", record)
```

```
            print("I/O Operations:", io_operations)
```

```
            return io_operations
```

```
        print("Record not found")
```

```
    return io_operations
```

```
# Create Index
```

```
index = {}
```

```
for block_no, block in enumerate(blocks, start=1):
```

```
    for record in block:
```

```
        index[record[0]] = block_no
```

```
print("\nIndex Table:")
```

```
for key, block_no in index.items():
```

```
    print(key, "-> Block", block_no)
```

```
# Indexed Retrieval
```

```
def indexed_retrieval(key):
```

```
    io_operations = 0
```

```
    # Access index
```

```
    io_operations += 1
```

```
    if key in index:
```

```
        block_no = index[key]
```

```
        # Access data block
```

```
        io_operations += 1
```

```
        for record in blocks[block_no - 1]:
```

```
            if record[0] == key:
```

```
                print("\nIndexed Retrieval")
```

```
                print("Record found:", record)
```

```
                print("I/O Operations:", io_operations)
```

```
                return io_operations
```

```
    print("Record not found")
```

```
    return io_operations
```

```
# Search for record
```

```

key = int(input("\nEnter record ID to search: "))

seq_io = sequential_retrieval(key)

index_io = indexed_retrieval(key)

# Comparison

print("\n--- Comparison ---")

print("Sequential I/O Operations:", seq_io)

print("Indexed I/O Operations:", index_io)

if seq_io < index_io:

    print("Sequential retrieval requires fewer I/O operations.")

elif index_io < seq_io:

    print("Indexed retrieval requires fewer I/O operations.")

else:

    print("Both methods require the same number of I/O operations.")

```

Output:

```

Secondary Storage Blocks:
Block 1 : [(101, 'Alice'), (102, 'Bob')]
Block 2 : [(103, 'Charlie'), (104, 'David')]
Block 3 : [(105, 'Eva'), (106, 'Frank')]
Block 4 : [(107, 'George'), (108, 'Helen')]

Index Table:
101 -> Block 1
102 -> Block 1
103 -> Block 2
104 -> Block 2
105 -> Block 3
106 -> Block 3
107 -> Block 4
108 -> Block 4

Enter record ID to search: 107

Sequential Retrieval
Record found: (107, 'George')
I/O Operations: 4

Indexed Retrieval
Record found: (107, 'George')
I/O Operations: 2

--- Comparison ---
Sequential I/O Operations: 4
Indexed I/O Operations: 2
Indexed retrieval requires fewer I/O operations.
>

```

Question 9 : Write a program to simulate secondary storage block access. Calculate the number of I/O operations for sequential vs. indexed retrieval.

Aim

To implement dense and sparse indexes on a sorted file and compare their search performance using timing analysis.

Algorithm

1. Create a sorted file and divide the records into fixed-size blocks.
2. Build a dense index containing every record key and a sparse index containing only the first key of each block.
3. Search for multiple keys using both indexes.
4. Measure and record the execution time for dense and sparse index searches.
5. Compare the timings and display which index provides faster search performance.

Code :

INPUT:

```
# Sparse index timing

start = time.perf_counter()

for key in search_keys:

    sparse_search(key)

sparse_time = time.perf_counter() - start

# -----

# Display Results

# -----
```

```
print("----- Index Information -----")

print("Total records      :", len(records))

print("Block size        :", block_size)

print("Dense index entries :", len(dense_index))

print("Sparse index entries:", len(sparse_index))

print("\n----- Search Example -----")

key = int(input("Enter student ID to search: "))

print("\nDense Index Result:")

print(dense_search(key))

print("\nSparse Index Result:")

print(sparse_search(key))

print("\n----- Timing Analysis -----")

print("Number of searches    :", len(search_keys))

print("Dense Index Time       :", dense_time, "seconds")

print("Sparse Index Time      :", sparse_time, "seconds")

if dense_time < sparse_time:

    print("\nDense index is faster.")

else:

    print("\nSparse index is faster.")

print("\nTime Difference:"
```


abs(dense_time - sparse_time), "seconds"

OUTPUT:

```
-----  
Secondary Storage Blocks:  
Block 1 : [(101, 'Alice'), (102, 'Bob')]  
Block 2 : [(103, 'Charlie'), (104, 'David')]  
Block 3 : [(105, 'Eva'), (106, 'Frank')]  
Block 4 : [(107, 'George'), (108, 'Helen')]  
  
Index Table:  
101 -> Block 1  
102 -> Block 1  
103 -> Block 2  
104 -> Block 2  
105 -> Block 3  
106 -> Block 3  
107 -> Block 4  
108 -> Block 4  
  
----- Index Information -----  
Total records      : 10000  
Block size        : 100  
Dense index entries : 10000  
Sparse index entries: 100  
  
----- Search Example -----  
Enter student ID to search: 7856  
  
Dense Index Result:  
(7856, 'Student_7856')  
  
Sparse Index Result:  
(7856, 'Student_7856')  
  
----- Timing Analysis -----  
Number of searches      : 1000  
Dense Index Time       : 0.00028110004495829344 seconds  
Sparse Index Time      : 0.006668300018645823 seconds  
  
Dense index is faster.  
  
Time Difference: 0.0063871999736875296 seconds  
|
```

Question : 10 Write a program to construct and traverse a B+ Tree, then execute a range query for keys between 15 and 45, showing all matching records.

Aim

To construct and traverse a B+ Tree, and perform a range query to display all records with keys between 15 and 45.

Algorithm

1. Create a B+ Tree with leaf and internal nodes.
2. Insert the given keys into the B+ Tree and split nodes when necessary.
3. Traverse the tree and display its structure.
4. Follow the linked leaf nodes to find keys from 15 to 45.
5. Display all matching records.

CODE:

INPUT:

B+ Tree Implementation

```
class Node:
```

```
    def __init__(self, leaf=False):
```

```
        self.keys = []
```

```
        self.children = []
```

```
        self.leaf = leaf
```

```
        self.next = None
```

```
class BPlusTree:
```

```
    def __init__(self, order=4):
```

```
        self.root = Node(leaf=True)
```

```
        self.order = order
```

```
    # Insert a key
```

```
    def insert(self, key, value):
```

```
        root = self.root
```

```
        if len(root.keys) == self.order - 1:
```

```
            new_root = Node()
```

```

new_root.children.append(root)

self.split_child(new_root, 0)

self.root = new_root

self.insert_non_full(self.root, key, value)

# Insert into non-full node

def insert_non_full(self, node, key, value):

    if node.leaf:

        i = 0

        while i < len(node.keys) and node.keys[i] < key:

            i += 1

        node.keys.insert(i, key)

        node.children.insert(i, value)

    else:

        i = 0

        while i < len(node.keys) and key >= node.keys[i]:

            i += 1

        if len(node.children[i].keys) == self.order - 1:

            self.split_child(node, i)

```

```

        if key >= node.keys[i]:

            i += 1

        self.insert_non_full(node.children[i], key, value)

# Split child

def split_child(self, parent, index):

    child = parent.children[index]

    new_node = Node(leaf=child.leaf)

    mid = len(child.keys) // 2

    if child.leaf:

        new_node.keys = child.keys[mid:]

        new_node.children = child.children[mid:]

        child.keys = child.keys[:mid]

        child.children = child.children[:mid]

    # Link leaf nodes

    new_node.next = child.next

    child.next = new_node

parent.keys.insert(index, new_node.keys[0])

```

```
parent.children.insert(index + 1, new_node
```

```
else:
```

```
    middle_key = child.keys[mid]
```

```
    new_node.keys = child.keys[mid + 1:]
```

```
    new_node.children = child.children[mid + 1:]
```

```
    child.keys = child.keys[:mid]
```

```
    child.children = child.children[:mid + 1]
```

```
    parent.keys.insert(index, middle_key)
```

```
    parent.children.insert(index + 1, new_node)
```

```
# Traverse tree
```

```
def traverse(self, node=None, level=0):
```

```
    if node is None:
```

```
        node = self.root
```

```
    print("Level", level, ":", node.keys)
```

```
    if not node.leaf:
```

```
        for child in node.children:
```

```
            self.traverse(child, level + 1)
```

```
# Range query
```

```
def range_query(self, start, end):
```

```
    node = self.root
```

```
    # Find first leaf
```

```
    while not node.leaf:
```

```
        i = 0
```

```
        while i < len(node.keys) and start >= node.keys[i]:
```

```
            i += 1
```

```
        node = node.children[i]
```

```
    result = []
```

```
    # Traverse linked leaves
```

```
    while node:
```

```
        for i in range(len(node.keys)):
```

```
            if start <= node.keys[i] <= end:
```

```
                result.append((node.keys[i], node.children[i]))
```

```
            if node.keys[i] > end:
```

```
                return result
```

```
node = node.next
```

```
return result
```

```
# -----
```

```
# Main Program
```

```
# -----
```

```
tree = BPlusTree(order=4)
```

```
# Key-value records
```

```
records = {
```

```
    10: "Record-10",
```

```
    15: "Record-15",
```

```
    20: "Record-20",
```

```
    25: "Record-25",
```

```
    30: "Record-30",
```

```
    35: "Record-35",
```

```
    40: "Record-40",
```

```
    45: "Record-45",
```

```
    50: "Record-50",
```

```
    55: "Record-55"
```

```
}
```

```
# Insert records
```

```
for key, value in records.items():
```

```
    tree.insert(key, value)
```

```
# Traverse B+ Tree
```

```
print("B+ Tree Traversal:")
```

```
tree.traverse()
```

```
# Range query
```

```
print("\nRange Query: 15 to 45")
```

```
result = tree.range_query(15, 45)
```

```
for key, value in result:
```

```
    print(key, "->", value)
```

OUTPUT:

```
Secondary Storage Blocks:
Block 1 : [(101, 'Alice'), (102, 'Bob')]
Block 2 : [(103, 'Charlie'), (104, 'David')]
Block 3 : [(105, 'Eva'), (106, 'Frank')]
Block 4 : [(107, 'George'), (108, 'Helen')]

Index Table:
101 -> Block 1
102 -> Block 1
103 -> Block 2
104 -> Block 2
105 -> Block 3
106 -> Block 3
107 -> Block 4
108 -> Block 4
B+ Tree Traversal:
Level 0 : [20, 30]
Level 1 : [15]
Level 2 : [10]
Level 2 : [15]
Level 1 : [25]
Level 2 : [20]
Level 2 : [25]
Level 1 : [35, 40, 45]
Level 2 : [30]
Level 2 : [35]
Level 2 : [40]
Level 2 : [45, 50, 55]

Range Query: 15 to 45
15 -> Record-15
20 -> Record-20
25 -> Record-25
30 -> Record-30
35 -> Record-35
40 -> Record-40
45 -> Record-45
```